Software Requirements Specification

Restaurant App MVP (Frontend Only, React Native)

Assignment 1 — Mobile Application Development — Fall 2026

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Restaurant App MVP, a React Native mobile application that lets diners browse a restaurant's menu, search and filter dishes, manage a shopping cart, reserve a table, place an order, and track that order in real time, while a Restaurant Manager reviews incoming orders, manages reservations, and maintains the menu. This SRS covers Version 1.0 (the MVP), which is scoped to a frontend-only implementation with no live backend. The intended audience is the course instructor/evaluator, the two-person development team (acting as both developers and their own client during the self-interview described in the assignment brief), and any future contributor who extends the app with a real backend.

1.2 Scope

In scope for this initial version:

- Customer sign-up and login against a local mock user list, with role selection (Customer or Manager).
- Browsing the full menu, organised into categories (Starters, Mains, Desserts, Drinks), with a "Daily Special" badge and greyed-out unavailable items.
- Searching the menu by name/description with debounced input, plus category filtering and sorting by price or name.
- Adding items to a cart, adjusting quantities, adding per-item special instructions, and applying a mock promo code for a percentage discount.
- Viewing an order summary with subtotal, service charge, sales tax, discount and grand total, computed from named rate constants.
- Reserving a table for a given date, time slot and party size, with availability checked against a local table/reservation dataset, and a "My Reservations" list supporting cancellation.
- Placing an order (dine-in or takeaway) and watching it progress automatically through Pending → Preparing → Ready → Served on a demo timer.
- A Manager Dashboard, visible only to accounts with the Manager role, for updating order status, accepting/declining reservations, and adding items / editing prices / toggling availability on the shared menu.
- Toggling between light and dark themes app-wide, and persisting session, orders, and menu edits on the device using AsyncStorage.

Out of scope for this version:

- Real payments or any payment gateway integration.
- Push notifications of any kind.
- Any backend server, REST/GraphQL API, or live database — all data is mock JSON/JS held in React state and (where noted) AsyncStorage.
- Real-time multi-device sync between a customer's phone and the manager's phone (each device holds its own local copy of the mock data).
- Third-party state-management libraries (Redux, MobX, Zustand, etc.) — only React's built-in hooks and Context API are used.

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification — this document, defining what the system must do.
MVP	Minimum Viable Product — the smallest feature set that delivers a usable, demonstrable product.
UML	Unified Modeling Language — the diagram notation used in Question 2 (use case, class, sequence, state, component).
Hook	A React function (e.g. useState, useEffect) that lets a function component use state or lifecycle features.
Context	React's built-in mechanism (createContext/useContext) for sharing state across a component tree without prop drilling.
Reducer	A pure function (state, action) => newState, used with useReducer to manage complex, related state transitions.
Mock Data	Static, hand-authored JSON/JavaScript data that stands in for a real backend response in this frontend-only build.
FlatList	React Native's performant, virtualised list component, used to render the menu, cart, and dashboard lists.
AsyncStorage	A React Native library providing simple, asynchronous, persistent key-value storage on the device.
Debounce	A technique that delays acting on rapidly-changing input (e.g. keystrokes) until it pauses for a set interval.

2. Overall Description

2.1 User Roles

Role	Key Permissions & Goals
Customer	Sign up/log in; browse, search, and filter the menu; add items to cart and apply promo codes; reserve a table; place dine-in or takeaway orders; track order status; toggle theme; view/edit own profile.
Restaurant Manager	Everything a Customer can view, plus: access the Manager Dashboard; move any order through its status lifecycle or cancel it; accept or decline table reservations; add new menu items, edit prices, and toggle item availability, with changes reflected on the customer menu immediately.

2.2 User Stories

Derived from the self-interview described in the assignment brief ("As a Customer, I'm hungry and in a hurry…" / "As a Restaurant Manager, I struggle with phone-in orders…").

Customer stories (from the self-interview):

1. As a Customer, I want to browse today's specials at a glance, so that I can decide quickly what to order without reading the whole menu.
2. As a Customer, I want to search the menu by name, so that I can jump straight to a dish I already have in mind.
3. As a Customer, I want to reserve a table for a specific time, so that my food is ready when I arrive and I skip the queue.
4. As a Customer, I want to add special instructions to an item (e.g. "no onions"), so that the kitchen prepares it the way I actually want to eat it.
5. As a Customer, I want to apply a promo code at checkout, so that I can save money on my order.
6. As a Customer, I want to track my order's status after placing it, so that I know exactly when to expect my food.
7. As a Customer, I want to switch to dark mode, so that the app is comfortable to use at night.

Manager stories:

8. As a Restaurant Manager, I want to see all incoming orders in one place, so that I no longer rely on error-prone phone-in orders.
9. As a Restaurant Manager, I want to update an order's status myself, so that I can correct or speed up the automatic progression when the kitchen actually finishes early or is delayed.
10. As a Restaurant Manager, I want to edit a menu item's price or mark it unavailable, so that the digital menu always matches what the kitchen can actually serve today.
11. As a Restaurant Manager, I want to accept or decline table reservations, so that I can manage seating capacity efficiently.

3. Functional Requirements

Authentication

12. FR-01: The system shall allow a new user to sign up with full name, email, password, confirm password, and a role selection (Customer or Manager).
13. FR-02: The system shall validate that the email matches a standard email format before allowing sign-up or login.
14. FR-03: The system shall require a password of at least eight characters containing at least one digit.
15. FR-04: The system shall authenticate returning users against the mock users dataset and reject invalid credentials with a clear on-screen error.
16. FR-05: The system shall persist the logged-in user's session in AsyncStorage so the user remains logged in after closing and reopening the app.

Menu

17. FR-06: The system shall load the menu from a simulated network call and display a loading indicator while it is in progress.
18. FR-07: The system shall display an error message with a Retry button if the simulated menu load fails.
19. FR-08: The system shall group menu items into categories and allow the user to filter the visible items by selecting a category chip.
20. FR-09: The system shall visually badge items marked as a "Daily Special".
21. FR-10: The system shall grey out unavailable items and disable their "Add" button.
22. FR-11: The system shall support pull-to-refresh on the menu list.
23. FR-12: The system shall allow the user to sort the menu by price (ascending/descending) or name (A–Z).

Search

24. FR-13: The system shall let the user search the menu by item name or description, applying the filter after 400ms of typing inactivity (debounced).
25. FR-14: The system shall display the user's five most recent distinct search terms as tappable suggestions when the search field is focused and empty.
26. FR-15: The system shall display a friendly empty-state message when no item matches the current search/filter combination.

Cart

27. FR-16: The system shall allow the user to add a menu item to the cart, incrementing its quantity if it is already present.
28. FR-17: The system shall allow the user to increment, decrement, or remove a cart item, removing it automatically when its quantity reaches zero.
29. FR-18: The system shall allow the user to attach a free-text special-instructions note to each cart item.

30. FR-19: The system shall allow the user to apply a promo code, applying the associated percentage discount if valid or showing an error if not.
31. FR-20: The system shall display a live badge on the cart icon showing the total quantity of items in the cart.

Reservation

32. FR-21: The system shall let the user choose a date, an hourly time slot between 12:00 and 22:00, and a party size between 1 and 12.
33. FR-22: The system shall disable a time slot when no table with sufficient seating is free at that date and time.
34. FR-23: The system shall validate that the phone number matches the Pakistani mobile format (03XX-XXXXXXX) and that the booking is at least one hour in the future.
35. FR-24: The system shall show a confirmation modal summarising the booking before it is saved.
36. FR-25: The system shall let the user cancel an existing reservation after an on-screen confirmation.

Orders

37. FR-26: The system shall let the user choose Dine-in (with a table) or Takeaway (with a pickup time) before placing an order.
38. FR-27: The system shall compute subtotal, service charge, sales tax, promo discount, and grand total using named, constant rates.
39. FR-28: The system shall automatically progress a placed order's status from Pending to Preparing, Ready, and Served on fixed demo timers.
40. FR-29: The system shall display a running elapsed-time counter and a step progress indicator on the Order Tracking screen.
41. FR-30: The system shall persist all placed orders in AsyncStorage and reload them when the app restarts.

Manager Dashboard

42. FR-31: The system shall restrict access to the Manager Dashboard to users whose role is Manager.
43. FR-32: The system shall let the Manager manually change any order's status from an Incoming Orders list.
44. FR-33: The system shall let the Manager accept or decline a pending table reservation.
45. FR-34: The system shall let the Manager add a new menu item, edit an existing item's price, or toggle its availability, with the change reflected on the Customer's menu screen immediately through shared state.

4. Non-Functional Requirements

Usability

- All interactive controls (buttons, chips, switches) must have a minimum touch target of 40×40 dp and clear visual feedback on press.
- Form validation errors must appear inline, next to the field they concern, and must clear as soon as the user edits that field.

Performance

- The menu screen's initial "load" (simulated fetch) must resolve within 1.5–2 seconds and show a loading indicator throughout.
- Scrolling a FlatList of 15+ items must remain smooth (no visible frame drops) by using keyExtractor, memoised row components, and avoiding inline function props where they would defeat memoisation.

Responsiveness

- All screens must render correctly on common phone widths (360dp–430dp) using flexbox layouts and relative sizing rather than fixed pixel widths.

- Safe-area insets must be respected on notched devices so no interactive control sits under a system bar.

Maintainability

- Source code is organised under src/ into components, screens, context, reducers, hooks, data, and navigation, so a new contributor can locate any piece of logic by its role.
- Shared UI patterns (cards, chips, buttons) are implemented once and reused rather than duplicated per screen.

Data Handling (no backend)

- All catalogue data (menu, tables, users, promo codes) ships as static JS modules under src/data and is loaded into React state on demand.
- Data that must survive an app restart (session, orders, menu edits) is persisted to AsyncStorage and rehydrated on launch, with a loading screen shown until rehydration completes.

5. Client-Side Data Model (Mock Data)

Because no database is used, the following datasets are stored as local JavaScript modules and held in React state (via Context/reducers) at runtime.

Data Set	Fields	Description
Users	id, fullName, email, password, role	Mock accounts used by LoginScreen to simulate authentication; role is "customer" or "manager".
Categories	name (string)	The four menu groupings — Starters, Mains, Desserts, Drinks — used to render filter chips.
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	The restaurant's catalogue of dishes; edited by the Manager and read by the Customer via shared MenuContext.
Tables	id, label, seats	Physical tables consulted by useReservation to compute per-slot availability.
Reservations	id, userId, tableId, date, time, partySize, contactPhone, status	Bookings created via ReservationScreen; status is confirmed or cancelled.
Orders	id, items, total, type, table, pickupTime, status, createdAt	Placed orders managed by OrdersContext/ordersReducer; status moves through Pending → Preparing → Ready → Served (or Cancelled).

6. UML Diagrams

The five diagrams below were produced for Question 2 and model the system before implementation began.

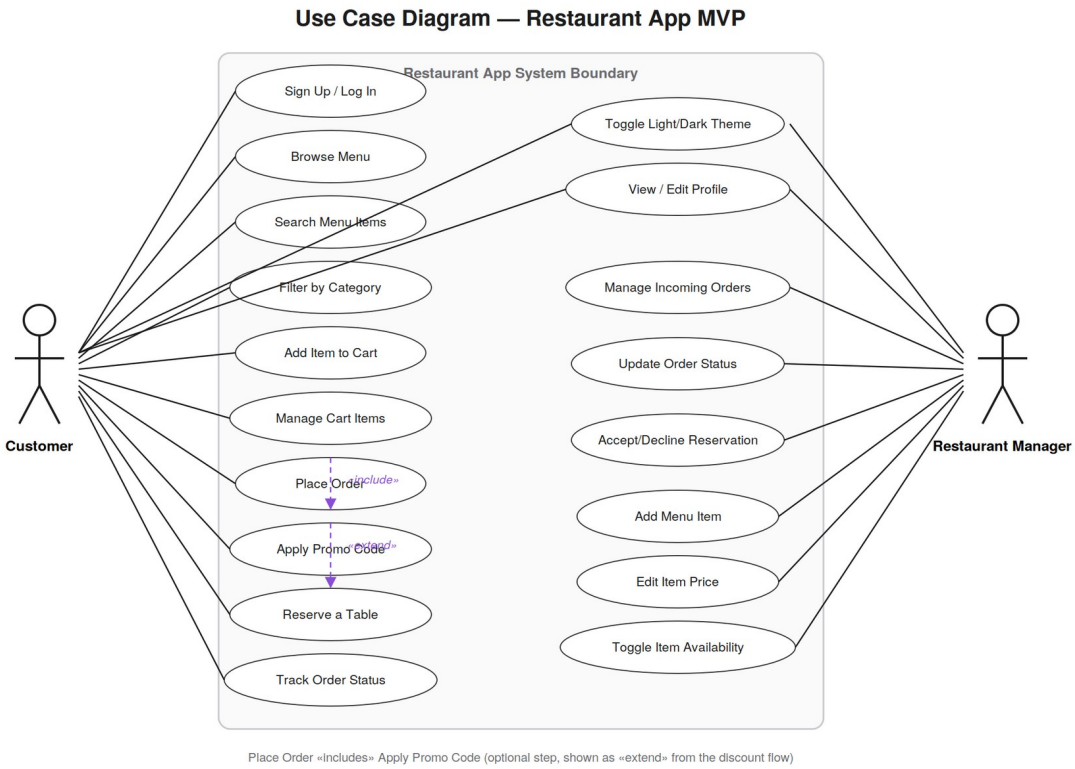


Figure 1 — Use Case Diagram: Customer and Restaurant Manager actors and their interactions with the system.

This diagram captures every top-level interaction a Customer or Manager can have with the app. The Customer side covers the full ordering journey — from signing up, through browsing/searching/filtering the menu, to cart management, promo codes, reservations, and order tracking. The Manager side covers dashboard-only capabilities: managing incoming orders, reservations, and the shared menu catalogue. "Place Order" is modelled with an «include» relationship to the underlying checkout flow and an «extend» relationship to the optional "Apply Promo Code" step, since a promo code is not required to complete an order.

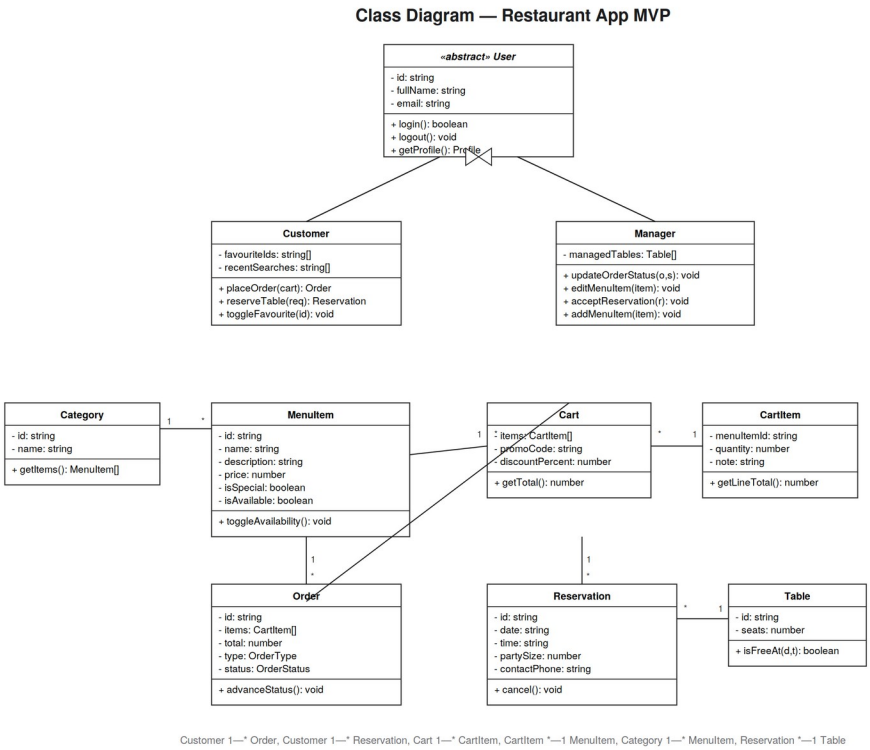


Figure 2 — Class Diagram: core domain classes, attributes, operations, and relationships.

User is modelled as an abstract base class, with Customer and Manager as concrete subclasses that inherit id/fullName/email and add their own role-specific data and operations — this mirrors how AuthContext stores one user object whose role field determines which subclass-like behaviour (placing orders vs. managing the dashboard) is exposed in the UI. A Category has many MenuItems; a Cart aggregates many CartItems, each referencing one MenuItem; a Customer can have many Orders and many Reservations; and each Reservation references exactly one Table.

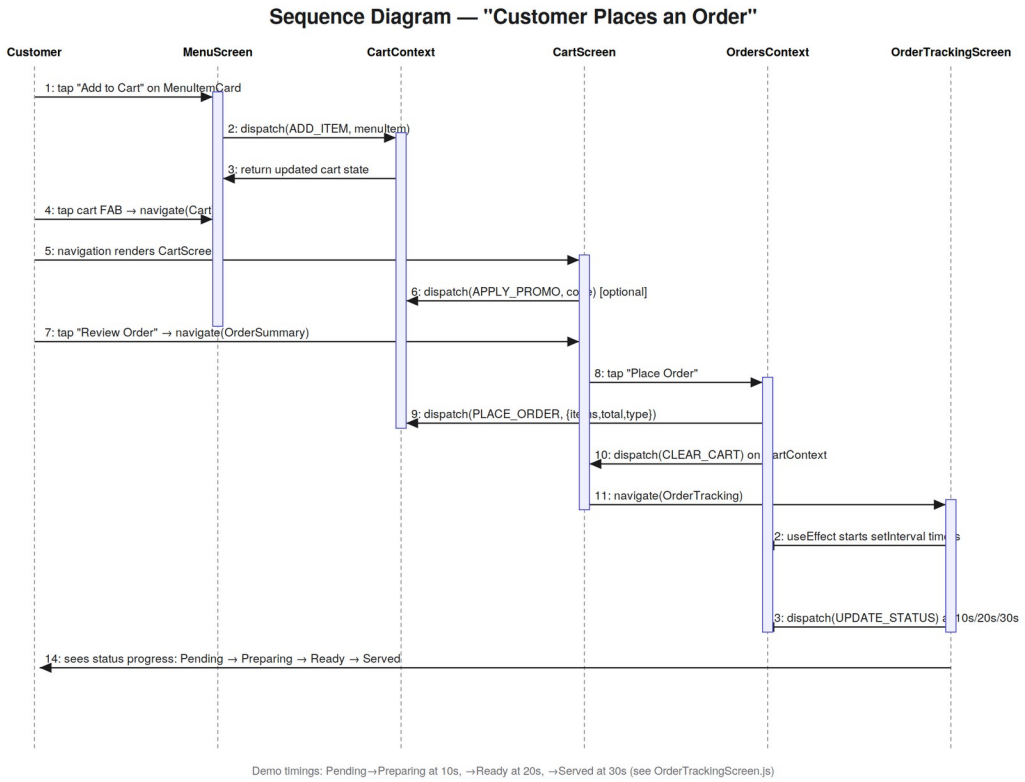


Figure 3 — Sequence Diagram: "Customer places an order", from Add to Cart to the Order Tracking screen.

The sequence begins the moment the Customer taps "Add to Cart" on a MenuItemCard, which dispatches ADD_ITEM to CartContext. It follows the Customer through reviewing the cart, optionally applying a promo code, confirming the order summary, and placing the order — which dispatches PLACE_ORDER to OrdersContext, clears the cart, and navigates to Order Tracking, where useEffect-driven timers begin advancing the order's status automatically.

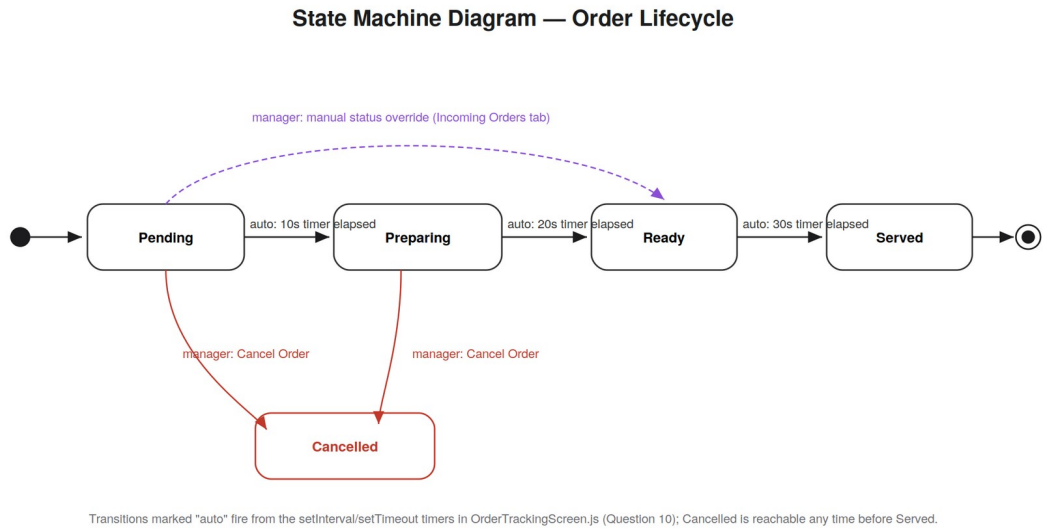


Figure 4 — State Machine Diagram: the Order lifecycle.

An order starts in Pending and advances automatically through Preparing, Ready, and Served on the 10/20/30-second demo timers implemented with setInterval/setTimeout in OrderTrackingScreen.js. At any point before

Served, the Manager can cancel the order from the Incoming Orders tab, moving it directly to a terminal Cancelled state; the Manager can also manually override the automatic progression at any time.

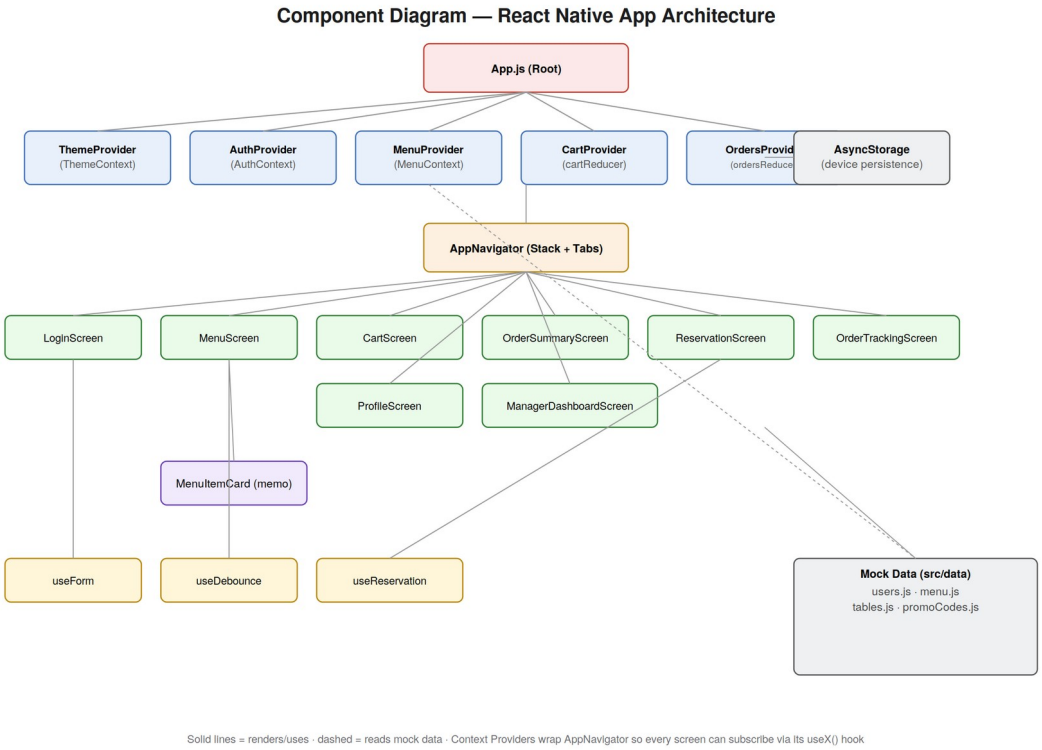


Figure 5 — Component Diagram: how screens, Context providers, reducers, and custom hooks depend on one another.

App.js composes five Context providers (Theme, Auth, Menu, Cart, Orders) around a single AppNavigator, so every screen underneath can subscribe to exactly the state it needs via its corresponding useX() hook, with no prop drilling. Three custom hooks (useForm, useDebounce, useReservation) extract reusable logic out of LoginScreen, MenuScreen, and ReservationScreen respectively, keeping those screens focused on presentation. All static content ultimately traces back to the mock data modules under src/data.

7. MVP Frontend Development (React Native)

For each screen below: purpose, UI elements, navigation entry/exit points, local data used, and the hooks used and why.

7.1 Login and Signup Screen

- Purpose: authenticate an existing user or register a new one, then route them to the correct home screen by role.
- UI elements: mode toggle (Login/Signup), controlled text inputs (email, password, full name, confirm password), show/hide password toggle, role selector chips, submit button with ActivityIndicator, inline error text per field.
- Navigation: entry point — the only screen shown when no user is logged in (Stack root). Exit — navigation.reset() to MainTabs (Customer) or ManagerDashboard (Manager) on success.
- Local data used: src/data/users.js (mock credential list).
- Hooks used: useState (mode, showPassword) directly in the screen; useForm (custom, Question 9) for values/errors/handleChange/handleSubmit/isSubmitting, since login and signup share the same controlled-input and validation shape.

7.2 Menu Browsing Screen

- Purpose: the customer's main hub — browse, filter, search, and sort the live menu, and add items to the cart.
- UI elements: search bar, category chips, sort options, FlatList of MenuItemCard components (image, badge, price, Add button, favourite heart), pull-to-refresh, floating "Back to top" and "Cart" buttons, loading/error states.
- Navigation: entry point — default tab after login for Customers. Exit — navigate to Cart (via the cart FAB) or implicitly stays as the tab home.
- Local data used: MenuContext (shared, backed by src/data/menu.js and manager edits).
- Hooks used: useState (category, sort, favourites, search text), useEffect (search-term tracking, header title), useRef (search input focus, FlatList ref, render counter, previous-query tracker), useMemo (derived filteredItems), useCallback (stable add/favourite handlers for MenuItemCard's React.memo), useDebounce (custom) and useMenu (custom, via MenuContext).

7.3 Search and Scroll Controls

- Purpose: make the Menu screen's search and long-list scrolling feel responsive and low-friction (this is implemented as an enhancement layer on MenuScreen rather than a separate screen).
- UI elements: focusable/clearable search TextInput, recent-search suggestion chips, floating "Back to top" button, small debug render-count label.
- Navigation: none — behaviour lives entirely within the Menu screen.
- Local data used: in-memory recent-searches array (component state, not persisted).
- Hooks used: useRef for the TextInput ref, the FlatList ref, the render counter, and the previous-query tracker (values that must persist across renders without themselves causing one); useState for the "show back-to-top" flag, which does need to trigger a re-render.

7.4 Profile and Theme Screen

- Purpose: show the logged-in user's identity and let them switch the whole app between light and dark mode.
- UI elements: avatar initial, name, email, role badge, dark-mode Switch, Log Out button.
- Navigation: entry point — Profile tab. Exit — navigation.reset() to Login on logout.
- Local data used: none of its own; reads the current user and theme from Context.
- Hooks used: useContext via useAuth() and useTheme(), since the same user/theme values are needed on every screen and prop drilling them down through the tab/stack navigators would be impractical.

7.5 Cart Screen

- Purpose: review, adjust, and annotate the items chosen so far before checking out.
- UI elements: FlatList of cart lines (name, price, quantity stepper, remove button, note field), promo-code input and Apply button, subtotal, "Review Order" button.
- Navigation: entry point — from the Menu screen's cart FAB. Exit — navigate to OrderSummary.
- Local data used: CartContext (items, promoCode, discountPercent).
- Hooks used: useContext via useCart(), which wraps useReducer internally — chosen over plain useState because the cart's several related fields (items array, promo code, discount) must change together through well-defined, testable transitions (see Question 7's reducer test table).

7.6 Order Summary Screen

- Purpose: show the final cost breakdown and let the customer choose dine-in vs. takeaway before placing the order.
- UI elements: order-type chips, itemised rows (subtotal, discount, service charge, tax, grand total), "Place Order" button.

- Navigation: entry point — from Cart's "Review Order" button. Exit — navigate to OrderTracking after placing the order (which also clears the cart).
- Local data used: CartContext (read) and OrdersContext (write, via PLACE_ORDER).
- Hooks used: useState for the dine-in/takeaway toggle; useMemo for the four computed totals, which depend only on cart items and the discount percentage and should not be recalculated on unrelated re-renders.

7.7 Table Reservation Screen

- Purpose: let the customer book a table for a specific date/time/party size and manage their existing bookings.
- UI elements: date input, time-slot chips (disabled when unavailable), party-size stepper, available-table chips, phone input, confirmation modal, "My Reservations" list with cancel.
- Navigation: entry point — Reservations tab. Exit — none (self-contained); a confirmation Alert appears on success.
- Local data used: src/data/tables.js (seed tables/reservations), all wrapped by the useReservation hook.
- Hooks used: useReservation (custom, Question 9) owns all business logic — date/time/party-size/table/contact state, availability computation, validation, createReservation, and cancelReservation — so the screen component itself contains no business logic, only rendering.

7.8 Order Tracking and Manager Dashboard

- Purpose: (Tracking) show the customer their most recent order progressing live; (Dashboard) give the Manager full operational control over orders, reservations, and the menu.
- UI elements: (Tracking) step progress indicator, elapsed-time counter, item list, total. (Dashboard) three tabs — Incoming Orders (status buttons), Reservations (accept/decline), Menu Management (add item, inline price edit, availability Switch).
- Navigation: Order Tracking — entry from Order Summary after placing an order, also reachable as its own tab. Manager Dashboard — entry point for Manager-role users only (extra tab / stack screen), hidden entirely from Customers.
- Local data used: OrdersContext (orders + status), MenuContext (menu items + edits), a locally-seeded reservations list for the demo Reservations tab.
- Hooks used: useEffect with setInterval/setTimeout (and cleanup) to auto-advance order status and tick the elapsed-time counter; useMemo for the current step index; useState for the Dashboard's active-tab and per-row edit state; useContext throughout via useOrders()/useMenu(). This screen also integrates every earlier hook indirectly, since it consumes the Context providers built on useReducer, useState, and useEffect from Questions 6, 7, and 10.

End of Document

Prepared for Assignment 1 — Restaurant App MVP (Frontend Only, React Native), Fall 2026.